

# CLOUD COMPUTING: COMPUTE & VIRTUALIZATION

## Module 3: Compute, Virtualization, Containers & Serverless

Author: Eng. Carlos Andrés Sierra, M.Sc.  
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor  
Computer Engineering Program  
School of Engineering  
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL  
FRANCISCO JOSÉ DE CALDAS

# Outline

- 1 Computational Resources
- 2 Virtualization Fundamentals
- 3 Containers
- 4 Serverless & Orchestration

} → public cloud

aaS



# Outline

- 1 Computational Resources
- 2 Virtualization Fundamentals
- 3 Containers
- 4 Serverless & Orchestration



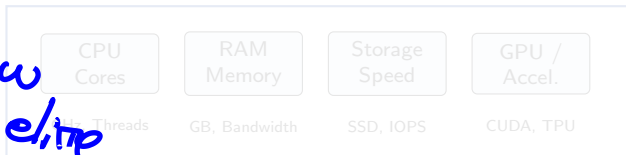
# What Defines *Computational Capacity*?

## The Four Pillars

Every workload requires **CPU**, **memory**, **storage**, and sometimes specialized accelerators. Mismatching any one creates bottlenecks.

*volatile / persistent*

flow  
pipeline  
document processing  
image



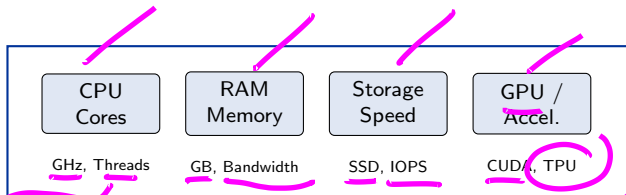
All matter: one bottleneck stalls everything



# What Defines *Computational Capacity*?

## The Four Pillars

Every workload requires **CPU**, **memory**, **storage**, and sometimes **specialized accelerators**. Mismatching any one creates bottlenecks.



**All matter: one bottleneck stalls everything**

*Parallelism*

*(dim, data, time)*

*Tensors*



# Performance Metrics

## Latency

Time to **respond** to a single request.  
Lower is better.

**Units:** ms,  $\mu$ s

## Throughput

Number of requests per unit time.  
Higher is better.

**Units:** req/s, Mbps

## IOPS

Input/Output operations per second.  
For *storage*.

Higher is better.

## Key Insight

Latency matters for **interactive** workloads (web, APIs). Throughput matters for **batch** workloads (data processing).



# Performance Metrics

## Latency

Time to **respond** to a single request.  
Lower is better.

**Units:** ms,  $\mu$ s

## Throughput

Number of requests **per unit time**.  
Higher is better.

**Units:** req/s, Mbps

## IOPS

Input/Output operations per second.  
For *storage*.  
Higher is better.

## Key Insight

Latency matters for **interactive** workloads (web, APIs). Throughput matters for **batch** workloads (data processing).



# Performance Metrics

## Latency

Time to **respond** to a single request.  
Lower is better.

**Units:** ms,  $\mu$ s

## Throughput

Number of requests **per unit time**.  
Higher is better.

**Units:** req/s, Mbps

## IOPS

Input/Output operations **per second**.  
For *storage*.

Higher is better.

## Key Insight

Latency matters for **interactive** workloads (web, APIs). Throughput matters for **batch** workloads (data processing).



# Performance Metrics

## Latency

Time to **respond** to a single request.  
Lower is better.

**Units:** ms,  $\mu$ s

## Throughput

Number of requests **per unit time**.  
Higher is better.

**Units:** req/s, Mbps

## IOPS

Input/Output operations **per second**.  
For *storage*.

Higher is better.

## Key Insight

**Latency** matters for **interactive** workloads (web, APIs). **Throughput** matters for **batch** workloads (data processing).



# Outline

- 1 Computational Resources
- 2 Virtualization Fundamentals**
- 3 Containers
- 4 Serverless & Orchestration



# The Consolidation Problem

## Traditional Model

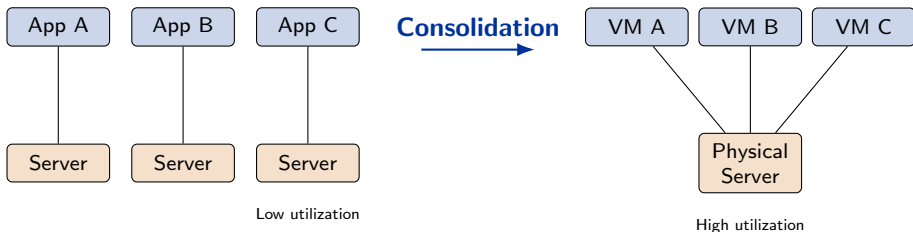
One application = one server. Result: **low utilization** and **high cost**.



# The Consolidation Problem

## Traditional Model

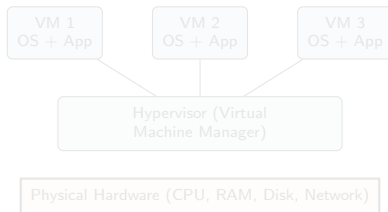
One application = one server. Result: **low utilization** and **high cost**.



# What is Virtualization?

## Core Idea

A **hypervisor** creates multiple **isolated** virtual machines (VMs) that share one physical server. Each VM thinks it owns the hardware.



## Key Components:

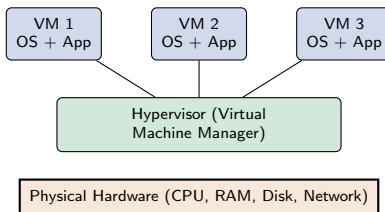
- vCPU: virtual processor assigned to VM
- vRAM: virtual memory pool
- vDisk: virtual storage volume



# What is Virtualization?

## Core Idea

A **hypervisor** creates multiple **isolated** virtual machines (VMs) that share one physical server. Each VM thinks it owns the hardware.



## Key Components:

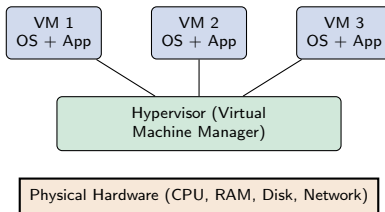
- vCPU: virtual processor assigned to VM
- vRAM: virtual memory pool
- vDisk: virtual storage volume



# What is Virtualization?

## Core Idea

A **hypervisor** creates multiple **isolated** virtual machines (VMs) that share one physical server. Each VM thinks it owns the hardware.



## Key Components:

- **vCPU**: virtual processor assigned to VM
- **vRAM**: virtual memory pool
- **vDisk**: virtual storage volume



# VM Isolation & Overcommitment

## Isolation

- VMs are **independent**
- Crash in one VM does **not** affect others
- **Resource limits**: guarantee fair share
- Example: VM1 can't starve VM2

## Overcommitment

- Allocate more vCPU than physical cores
- Works because not all VMs use 100%
- Risk: if all VMs peak, performance degrades
- Trade-off: cheaper vs. risky



# VM Isolation & Overcommitment

## Isolation

- VMs are **independent**
- Crash in one VM does **not** affect others
- **Resource limits**: guarantee fair share
- Example: VM1 can't starve VM2

## Overcommitment

- Allocate **more** vCPU than physical cores
- Works because not all VMs use 100
- Risk: if all VMs peak, performance degrades
- Trade-off: **cheaper** vs. **risky**



# Outline

- 1 Computational Resources
- 2 Virtualization Fundamentals
- 3 Containers**
- 4 Serverless & Orchestration



# The ‘‘It Works on My Machine’’ Problem

## Challenge

Application works on *your laptop* but *fails in production*. Why? Different OS, libraries, dependencies, configurations.

## Two Historical Solutions

- Virtual Machines: package entire OS + app
  - Solves problem
  - But: heavy, slow, expensive overhead
- Containers: package app + dependencies (no OS)
  - Same isolation as VM
  - But: lightweight, fast, minimal overhead



# The ‘‘It Works on My Machine’’ Problem

## Challenge

Application works on *your laptop* but *fails in production*. Why? Different OS, libraries, dependencies, configurations.

## Two Historical Solutions

### ① **Virtual Machines:** package entire OS + app

- Solves problem
- But: heavy, slow, expensive overhead

### ② **Containers:** package app + dependencies (no OS)

Same isolation as VM

• But: lightweight, fast, minimal overhead



# The ‘‘It Works on My Machine’’ Problem

## Challenge

Application works on *your laptop* but *fails in production*. Why? Different OS, libraries, dependencies, configurations.

## Two Historical Solutions

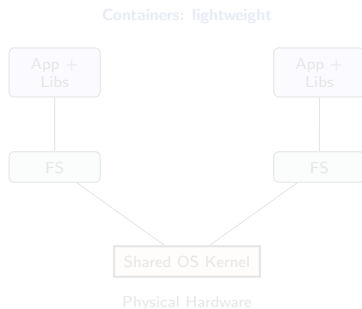
- ① **Virtual Machines:** package entire OS + app
  - Solves problem
  - But: heavy, slow, expensive overhead
- ② **Containers:** package app + dependencies (*no OS*)
  - Same isolation as VM
  - But: lightweight, fast, minimal overhead



# What is a Container?

## Definition

A **lightweight**, **isolated process** that includes an application, all dependencies, libraries, and configuration. Shares the *host OS kernel*.

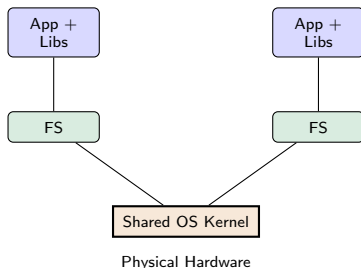


# What is a Container?

## Definition

A **lightweight**, **isolated process** that includes an application, all dependencies, libraries, and configuration. Shares the *host OS kernel*.

Containers: lightweight



# Container vs. VM

| Aspect         | Container          | VM              |
|----------------|--------------------|-----------------|
| Size           | 10–100 MB          | 1–10 GB         |
| Startup time   | ¡1 second          | 30–60 seconds   |
| Density        | 10s–1000s per host | 5–20 per host   |
| Isolation      | Process-level      | OS-level        |
| OS flexibility | Must match kernel  | Any OS          |
| Performance    | Near-native        | Slight overhead |
| Portability    | Excellent          | Good            |

## Bottom Line

Containers win on **efficiency and speed**. VMs win on **flexibility** (run Windows on Linux, etc.).



# Container vs. VM

| Aspect         | Container          | VM              |
|----------------|--------------------|-----------------|
| Size           | 10–100 MB          | 1–10 GB         |
| Startup time   | ¡1 second          | 30–60 seconds   |
| Density        | 10s–1000s per host | 5–20 per host   |
| Isolation      | Process-level      | OS-level        |
| OS flexibility | Must match kernel  | Any OS          |
| Performance    | Near-native        | Slight overhead |
| Portability    | Excellent          | Good            |

## Bottom Line

Containers win on **efficiency** and **speed**. VMs win on **flexibility** (run Windows on Linux, etc.).



# Outline

- 1 Computational Resources
- 2 Virtualization Fundamentals
- 3 Containers
- 4 Serverless & Orchestration



# The Scaling Challenge

## Problem

You have *10 containers*. As demand grows, should you deploy 50? 100? 1,000? **Who decides? Who manages them?**

## Two Answers:

### ● Orchestration: cluster of containers, automated scheduling

- You specify: *"I want 10 replicas"*
- Platform ensures: always 10 running (restart if fails)
- Example: Kubernetes

### ● Serverless: don't manage containers at all

- You write: a function that responds to events
- Platform runs it automatically, scales to zero
- Example: AWS Lambda, Google Cloud Functions



# The Scaling Challenge

## Problem

You have *10 containers*. As demand grows, should you deploy 50? 100? 1,000? **Who decides? Who manages them?**

## Two Answers:

### ❶ **Orchestration:** cluster of containers, automated scheduling

- You specify: *"I want 10 replicas"*
- Platform ensures: always 10 running (restart if fails)
- Example: Kubernetes

### ❷ **Serverless:** don't manage containers at all

- You write: a function that responds to events
- Platform runs it automatically, scales to zero
- Example: AWS Lambda, Google Cloud Functions



# The Scaling Challenge

## Problem

You have *10 containers*. As demand grows, should you deploy 50? 100? 1,000? **Who decides? Who manages them?**

## Two Answers:

- ① **Orchestration:** cluster of containers, automated scheduling
  - You specify: *"I want 10 replicas"*
  - Platform ensures: always 10 running (restart if fails)
  - Example: Kubernetes
- ② **Serverless:** don't manage containers at all
  - You write: a function that responds to events
  - Platform runs it automatically, scales to zero
  - Example: AWS Lambda, Google Cloud Functions



# What is Serverless?

## Definition

**Serverless** (*Functions as a Service, FaaS*) runs your code **only when triggered by an event**. No servers to manage, no idle costs.

## Characteristics:

- Event-driven: triggered by HTTP request, queue message, timer, etc.
- Auto-scaling: 0 → 1000 instances instantly (and back to 0)
- Pay-per-execution: only pay for execution time (Time → 100 ms → 100 ms)
- Stateless: first invocation is slow (cold start)
- Stateless: each invocation is independent

## Use Cases

Microservices, webhooks, scheduled jobs, image processing, API backends.

# What is Serverless?

## Definition

**Serverless** (*Functions as a Service*, FaaS) runs your code **only when triggered by an event**. No servers to manage, no idle costs.

## Characteristics:

- **Event-driven**: triggered by HTTP request, queue message, timer, etc.
- **Auto-scaling**: 0  $\rightarrow$  1000 instances instantly (and back to 0)
- **Pay-per-invocation**: only pay for execution time (100ms = 100 ms cost)
- **Cold starts**: first invocation is slow (*container boot*)
- **Stateless**: each invocation is independent

## Use Cases

Microservices, webhooks, scheduled jobs, image processing, API backends.

# What is Serverless?

## Definition

**Serverless** (*Functions as a Service, FaaS*) runs your code **only when triggered by an event**. No servers to manage, no idle costs.

## Characteristics:

- **Event-driven**: triggered by HTTP request, queue message, timer, etc.
- **Auto-scaling**:  $0 \rightarrow 1000$  instances instantly (and back to 0)
- **Pay-per-invocation**: only pay for execution time ( $100ms = 100\text{ ms cost}$ )
- **Cold starts**: first invocation is slow (*container boot*)
- **Stateless**: each invocation is independent

## Use Cases

Microservices, webhooks, scheduled jobs, image processing, API backends.

# What is Serverless?

## Definition

**Serverless** (*Functions as a Service, FaaS*) runs your code **only when triggered by an event**. No servers to manage, no idle costs.

## Characteristics:

- **Event-driven**: triggered by HTTP request, queue message, timer, etc.
- **Auto-scaling**:  $0 \rightarrow 1000$  instances instantly (and back to 0)
- **Pay-per-invocation**: only pay for execution time ( $100ms = 100\text{ ms cost}$ )
- Cold starts: first invocation is slow (*container boot*)
- Stateless: each invocation is independent

## Use Cases

Microservices, webhooks, scheduled jobs, image processing, API backends.

# What is Serverless?

## Definition

**Serverless** (*Functions as a Service*, FaaS) runs your code **only when triggered by an event**. No servers to manage, no idle costs.

## Characteristics:

- **Event-driven**: triggered by HTTP request, queue message, timer, etc.
- **Auto-scaling**:  $0 \rightarrow 1000$  instances instantly (and back to 0)
- **Pay-per-invocation**: only pay for execution time ( $100ms = 100\text{ ms cost}$ )
- **Cold starts**: first invocation is slow (*container boot*)
- **Stateless**: each invocation is independent

## Use Cases

Microservices, webhooks, scheduled jobs, image processing, API backends.

# What is Serverless?

## Definition

**Serverless** (*Functions as a Service*, FaaS) runs your code **only when triggered by an event**. No servers to manage, no idle costs.

## Characteristics:

- **Event-driven**: triggered by HTTP request, queue message, timer, etc.
- **Auto-scaling**:  $0 \rightarrow 1000$  instances instantly (and back to 0)
- **Pay-per-invocation**: only pay for execution time ( $100ms = 100\text{ ms cost}$ )
- **Cold starts**: first invocation is slow (*container boot*)
- **Stateless**: each invocation is independent

## Use Cases

Microservices, webhooks, scheduled jobs, image processing, API backends.

# What is Serverless?

## Definition

**Serverless** (*Functions as a Service*, FaaS) runs your code **only when triggered by an event**. No servers to manage, no idle costs.

## Characteristics:

- **Event-driven**: triggered by HTTP request, queue message, timer, etc.
- **Auto-scaling**:  $0 \rightarrow 1000$  instances instantly (and back to 0)
- **Pay-per-invocation**: only pay for execution time ( $100ms = 100\text{ ms cost}$ )
- **Cold starts**: first invocation is slow (*container boot*)
- **Stateless**: each invocation is independent

## Use Cases

Microservices, webhooks, scheduled jobs, image processing, API backends.

# Outline

- 1 Computational Resources
- 2 Virtualization Fundamentals
- 3 Containers
- 4 Serverless & Orchestration



## Recap: Module 3

- **Compute Resources:** CPU, RAM, Storage, GPU — all matter
- **Virtualization:** multiple VMs per physical server = cost savings
- **Containers:** lightweight alternative to VMs, faster deployment
- **Orchestration:** manage many containers automatically (Kubernetes)
- **Serverless:** don't manage infrastructure at all, pay per invocation
- **Choice:** depends on control vs. convenience trade-off



# Thanks!

## Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/cloud-computing>

